Topic 12: Dynamic Programming and Recursive Thinking

(How to solve infinite-dimensional problems one step at a time)

---

1. Why dynamic programming enters now

Topic 11 introduced discounting, infinite horizons, and the recursive decomposition

$$\bar{V} = (1-\delta)x + \delta\bar{V}.$$

That decomposition was for a constant payoff stream. But in most economic problems, payoffs are not constant — they depend on the **state** of the world, and on the **actions** of the agents. Often, those actions affect the future state as well.

For example, a firm's profit depends on its capital stock, which depends on past investment. A consumer's utility depends on her wealth, which depends on past savings decisions. A player's continuation payoff in a repeated game depends on the history of play, which determines the current "state" of the relationship.

In all these cases, the agent faces the same question every period: given the current state, what should I do today, knowing that my choice affects not only today's payoff but also tomorrow's state?

Dynamic programming is the method that turns this infinite-dimensional problem —choosing a full program of state-dependent choices—into a manageable one-step problem. The key idea is to find a **value function** that summarizes the future, so that the agent can optimize one period at a time.

---

2. The ingredients

A dynamic optimization problem has three components:

1. **State variable** $\theta \in \Theta$: a summary of everything from the past that matters for the future. The state might be a capital stock, a wealth level, a vector of past actions, or a belief.
2. **Action** $a \in A(\theta)$: what the agent chooses in the current period, given the state. The feasible set may depend on the state (you cannot invest more than your current wealth).
3. **Transition**: a rule that determines the next state θ' given the current state θ and action a. In the deterministic case this is a function $\theta' = g(\theta, a)$. In the stochastic case, θ' is drawn from a distribution that depends on (θ, a).

The agent's per-period payoff is $u(\theta, a)$, and she discounts the future with discount factor $\delta \in (0, 1)$.

The agent's problem is to choose a sequence of actions $(a_0, a_1, a_2, \dots)$ to maximize the normalized present value

$$(1-\delta)\sum_{t=0}^{\infty} \delta^t u(\theta_t, a_t),$$

where $\theta_{t+1} = g(\theta_t, a_t)$ and θ_0 is given.

This is, in principle, a problem with infinitely many choice variables. Dynamic programming reduces it to one.

---

3. The principle of optimality

The key insight is due to Bellman:

> "If a plan is optimal from period 0 onward, then its continuation from period 1 onward must be optimal given the state at period 1."

This sounds obvious, but it is powerful. It says that optimal behavior has a recursive structure: you cannot be doing the best thing overall without also doing the best thing going forward from every intermediate point.

The contrapositive makes the logic clear: if the continuation were not optimal from period 1, you could improve the overall plan by swapping in a better continuation — contradicting optimality of the original plan.

The principle of optimality is what justifies reducing the infinite-horizon problem to a one-period problem. Instead of choosing an entire sequence, the agent asks: "given the current state θ, what action a maximizes today's payoff plus the discounted value of being in state θ' tomorrow?" If we know the value of being in each state, the problem is solved.

---

4. The Bellman equation

The **value function** $V : \Theta \to \mathbb{R}$ assigns to each state θ the normalized value of behaving optimally from θ onward.

By the principle of optimality, V must satisfy the **Bellman equation**:

$$V(\theta) = \max_{a \in A(\theta)} \left\{ (1-\delta)\, u(\theta, a) + \delta\, V(g(\theta, a)) \right\}.$$

> "The value of being in state θ is the best you can do today — choosing an action that maximizes the weighted sum of today's payoff and the discounted value of tomorrow's state."

Although the Bellman equation does simplify the problem enormously, it, depending on the eye of the beholder, it is (un)fortunately not yet trivial. Why? Because the Bellman equation is a **functional equation**: the unknown is not a number but a *function* V. The value function appears on both sides of the equation, because the value of today's state depends on the value of tomorrow's state, which is itself determined by V.

Compare this to the simple recursive decomposition in Topic 11. There, the "state" was trivial (the payoff was the same every period), so V was just a number. Now V is a function of the state, and the Bellman equation tells us what this function must look like.

---

5. The Bellman equation as a fixed-point problem

The next step is now to solve this equation. In Topic 10, we studied fixed points: a point x^* such that $f(x^*) = x^*$. The Bellman equation has exactly this structure, but in a different space.

Define an *operator* T that takes a function $W : \Theta \to \mathbb{R}$ and produces a new function $TW : \Theta \to \mathbb{R}$ by

$$(TW)(\theta) = \max_{a \in A(\theta)} \left\{ (1-\delta)\, u(\theta, a) + \delta\, W(g(\theta, a)) \right\}.$$

Then the Bellman equation says: $V = TV$.

> "The value function is a fixed point of the Bellman operator. It is a function that, when you plug it into the right-hand side, you get back the same function on the left."

Let's reiterate how we get there.[1]

1. First, we notice that the agent's problem is described by the principle of optimality. She does whatever is best for her which means assuming that wherever she ends up tomorrow she will do what is best from tomorrow onward. Knowing that, today's choice boils down to choosing what happens today and where to end up tomorrow.
2. Second, using this "truth" we can determine what describes the value of starting at a particular state θ. It is the best that can come out from wherever we are. If we want to describe that **for all** possible states, we get the value function.
3. But then, this is *one single function*, completely pinned down by our inputs (state space, action space, transition function).

[1] My macro professor in the PhD classified people in two intelligence levels: Those that can solve Bellman equations and those that cannot. I do not subscribe to his view, but maybe it is still worth it to invest into understanding them, if only to impress a Macro Economist.

4. Finally, this means we can write the right hand side as a *function over functions* that for each value function W we pick on the right hand side (optimal or not) gives us the best action for today, that is,

$$T(W,\theta) = \max_{a \in A(\theta)} \{(1-\delta)\, u(\theta,a) + \delta\, W(g(\theta,a))\}$$

Notice that on the right hand side a is already pinned down by the max. So we are left with θ which enters through $W(g())$ and $u()$. So the only "choice" we have is that of W. If we can find a $W : \Theta \to \mathbb{R}$ such that $W(\theta) = T(W,\theta)$ for all θ, then we know that for this W the agent always takes the correct decision. But because θ is still an element here, we have to find our fixed point in the space of functions, not just numbers. This is what makes it a functional equation.

Our function space is the space of all (bounded) functions from Θ to $\mathbb{R}$.[2] The question is: does such a fixed point exist? Is it unique? And can we find it? The answer to all three is yes, thanks to a new fixed-point theorem.

---

6. The contraction mapping theorem

The fixed-point theorems in Topic 10 — Brouwer, Kakutani, Tarski — guarantee existence but not uniqueness (except Tarski for extremal fixed points). For the Bellman equation, we construct something stronger: a result that gives both existence and uniqueness, and tells us how to compute the solution.

The right tool is the **contraction mapping theorem** (also called the Banach fixed-point theorem).

Complete metric spaces

To state it, we need one concept. A **complete metric space** is a set X with a metric d (recall the remark in Topic 4) where every Cauchy sequence converges.[3] The real line $\mathbb{R}$ is complete (this is a fundamental property of the reals). The space of bounded functions from Θ to $\mathbb{R}$, equipped with the "sup metric" $d(f,g) = \sup_{\theta\in\Theta} |f(\theta) - g(\theta)|$, is also complete.[4]

The theorem

---

[2] If u is bounded, and $\delta < 1$, then any solution sequence is going to yield a bounded payoff, thus any function that could work must be a function that is bounded too.

[3] A Cauchy sequence is a sequence where the terms get arbitrarily close to each other as the sequence progresses: for every $\varepsilon > 0$, there exists N such that $d(x_m, x_n) < \varepsilon$ for all $m, n \geq N$. Completeness means such sequences actually have a limit in X — they don't "converge to a hole."

[4] The sup metric measures the largest pointwise difference between two functions. Two functions are close under this metric if they are close *everywhere*. An exercise to work with metrics is to prove that the sup metric is indeed a metric.

Theorem (Contraction Mapping / Banach). Let (X, d) be a complete metric space and let $T : X \to X$ be a **contraction**: there exists $\beta \in (0, 1)$ such that

$$d(T(x), T(y)) \leq \beta\, d(x, y) \quad \text{for all } x, y \in X.$$

Then T has exactly one fixed point x^*, and for any starting point $x_0 \in X$, the sequence $x_0, T(x_0), T(T(x_0)), \ldots$ converges to x^*.

> "A contraction squeezes points closer together. If you keep applying it, everything collapses to a single fixed point."

Proof sketch

The argument is constructive. Start anywhere: pick any $x_0 \in X$. Define $x_1 = T(x_0)$, $x_2 = T(x_1)$, and so on.

Because T is a contraction,

$$d(x_1, x_2) = d(T(x_0), T(x_1)) \leq \beta\, d(x_0, x_1).$$

Repeating,

$$d(x_n, x_{n+1}) \leq \beta^n\, d(x_0, x_1).$$

Since $\beta < 1$, these distances shrink geometrically — exactly like the geometric series from Topic 11. For any $m > n$, the triangle inequality gives

$$d(x_n, x_m) \leq d(x_n, x_{n+1}) + d(x_{n+1}, x_{n+2}) + \cdots \leq \beta^n\, d(x_0, x_1) \cdot \frac{1}{1 - \beta}.$$

As $n \to \infty$, the right hand side goes to 0, because $d(x_0, x_1)$ is bounded, because the functions are bounded, $1 - \beta > 0$ and $\beta^n \to 0$ because $\beta < 1$, so the sequence is Cauchy. By completeness, it converges to some x^*.

To see that x^* is a fixed point: T is continuous (any contraction is), so $T(x^*) = T(\lim x_n) = \lim T(x_n) = \lim x_{n+1} = x^*$.

Uniqueness: if there were two fixed points x^* and y^*, then $d(x^*, y^*) = d(T(x^*), T(y^*)) \leq \beta\, d(x^*, y^*)$. Since $\beta < 1$, this implies $d(x^*, y^*) = 0$, so $x^* = y^*$ because in every metric $d(x, y) = 0 \Leftrightarrow x = y$.

> "Start anywhere, keep applying the contraction, and you converge to the unique fixed point. The proof is the algorithm."

Comparison with Topic 10

The contraction mapping theorem has a very different character from Brouwer, Kakutani, and Tarski:

- it requires no convexity and no lattice structure,

- it requires a metric and completeness (topological structure that the others did not need),
- it gives **uniqueness** (the others generally do not),
- it is **constructive** (the iteration $x, T(x), T(T(x)), \ldots$ converges to the solution).

The price is the contraction property: the mapping must strictly squeeze distances. This is a strong requirement, but one that the Bellman operator naturally satisfies.

Another practical limitation is that value function iteration is often not very elegant. You can solve the problem, but the solution is often not closed-form, and — I am out of my depth here, but at least in my days — it is not very computationally efficient. That's why in Macro you often use other methods than value function iteration to determine the optimum. See more on this below.

---

7. Why the Bellman operator is a contraction

This is the result that makes everything work. Under standard assumptions — bounded payoffs, $\delta < 1$ — the Bellman operator T is a contraction on the space of bounded functions, with contraction factor $\beta = \delta$.

The intuition is direct. Suppose two candidate value functions W and W' differ by at most ε everywhere: $\sup_\theta |W(\theta) - W'(\theta)| \leq \varepsilon$. Now compute TW and TW'. The only way W and W' enter the Bellman equation is through the continuation value, which is multiplied by δ. So the difference between TW and TW' is at most $\delta\varepsilon$.[5]

This means:

$$d(TW, TW') \leq \delta \, d(W, W').$$

Since $\delta < 1$, the operator T is a contraction. The contraction mapping theorem then guarantees:

- the Bellman equation has a **unique** solution V,
- starting from any initial guess W_0 and iterating $W_{n+1} = TW_n$ converges to V.

> "Discounting is what makes the Bellman operator a contraction. Because the future is worth less than the present, disagreements about continuation values get squeezed by δ every time you iterate."

---

[5] More precisely: for any state θ and any action a, $|(1-\delta)u(\theta,a) + \delta W(g(\theta,a)) - (1-\delta)u(\theta,a) - \delta W'(g(\theta,a))| = \delta |W(g(\theta,a)) - W'(g(\theta,a))| \leq \delta\varepsilon$. Taking the max over a on each side and then the sup over θ gives $d(TW, TW') \leq \delta \, d(W, W')$.

8. Value iteration and policy functions

The constructive nature of the contraction mapping theorem gives us an algorithm: **value iteration**.

1. Start with any bounded function W_0 (a common choice is $W_0 = 0$).
2. Compute $W_1 = TW_0$, then $W_2 = TW_1$, and so on.
3. The sequence $W_0, W_1, W_2, \ldots$ converges to the true value function V.

At each step, computing TW_n requires solving a static optimization problem — maximizing over today's action — for each state. This is where the single-agent tools from Topics 2–7 come in: for each state, we solve a one-period problem. When we want the derivative of the resulting optimized value with respect to a state or parameter, Topic 7's envelope logic applies.

Once we have the value function V, the **policy function** (or policy correspondence) is the argmax of the Bellman equation:

$$a^*(\theta) = \text{argmax}_{a \in A(\theta)} \left\{ (1-\delta)\, u(\theta, a) + \delta\, V(g(\theta, a)) \right\}.$$

> "The policy function tells the agent what to do in each state. It is the decision rule that, together with the value function, solves the entire infinite-horizon problem."

This connects directly to Topic 5: the argmax is in general a correspondence (set-valued), and its properties — upper hemicontinuity, convexity of values — depend on the same assumptions that Berge's Maximum Theorem requires.

---

9. An economic example: optimal savings

Consider a consumer with wealth θ who must split it between consumption c and savings $\theta - c$ each period. She earns a gross return $R > 0$ on savings, so next period's wealth is $(\theta - c) \cdot R$. Her per-period utility from consumption is $u(c)$, increasing and concave.

The state is wealth θ. The action is how much to consume, $c \in [0, \theta]$. The transition is $\theta' = (\theta - c)R$.

The Bellman equation spells this out:

$$V(\theta) = \max_{0 \le c \le \theta} \left\{ (1-\delta)\, u(c) + \delta\, V((\theta - c)R) \right\}.$$

The consumer's trade-off is transparent: consuming more today (higher $u(c)$) means saving less and arriving at a lower wealth state tomorrow (lower $V((\theta - c)R)$). The weights $(1-\delta)$ and δ govern how she balances present enjoyment against future wealth.

Under standard assumptions (u bounded and continuous, $\delta R < 1$ to ensure the problem is well-behaved), the contraction mapping theorem guarantees a unique

value function V, and the policy function $c^*(\theta)$ tells us how much to consume at each wealth level.

This example shows dynamic programming in its most natural habitat: the consumer does not need to plan her entire life's consumption at once. She just needs to know V and solve a one-period problem. The value function encodes everything about the future that matters for today's decision.

---

10. Where this appears in Mas-Colell, Whinston, and Green (MWG)

Dynamic programming is not developed as a standalone topic in MWG's mathematical appendices. It appears in the economic chapters:

- **MWG, Chapter 20** (intertemporal choice and dynamic optimization),
- **MWG, Chapters 12 and 13** (repeated games, where continuation values play the role of the value function).

The contraction mapping theorem is stated in **MWG, Mathematical Appendix C**. The connection between the Bellman equation and the contraction mapping theorem is typically developed in more specialized references.[6]

---

11. Why this matters for microeconomics

Dynamic programming is how economists tame infinite-horizon problems.

It matters because:

- it converts infinite-dimensional optimization into a sequence of one-period problems,
- it provides both existence and uniqueness of optimal behavior (via the contraction mapping theorem),
- it gives a computational method (value iteration),
- and it underlies the recursive formulation of repeated games, dynamic contracts, and mechanism design.

In the PhD course, you will see continuation values used as incentive instruments — a principal can promise future rewards or threaten future punishments to influence behavior today. The Bellman equation is what makes "the value of the future" a precise, well-defined object that can be manipulated and optimized over.

[6] Stokey, Lucas, and Prescott, "Recursive Methods in Economic Dynamics" (1989), is the standard reference for economists. It develops the contraction mapping approach to dynamic programming in detail. Kreps (1995), "A Course in Microeconomic Theory," has a more micro oriented treatment.

12. What you should take away

After this topic, you should be comfortable with:

- reading Bellman equations and understanding what the value function represents,
- seeing the Bellman equation as a fixed-point problem in function space,
- knowing what the contraction mapping theorem says and why it gives both existence and uniqueness,
- understanding that $\delta < 1$ is what makes the Bellman operator a contraction,
- and recognizing that dynamic programming reduces infinite-horizon problems to sequences of static problems.

This is the last major mathematical tool. From here, the remaining topics refine and extend what we have already built.